CV802

Assignment 3: Novel-view Synthesis

Anas Khan

Problem Statement:

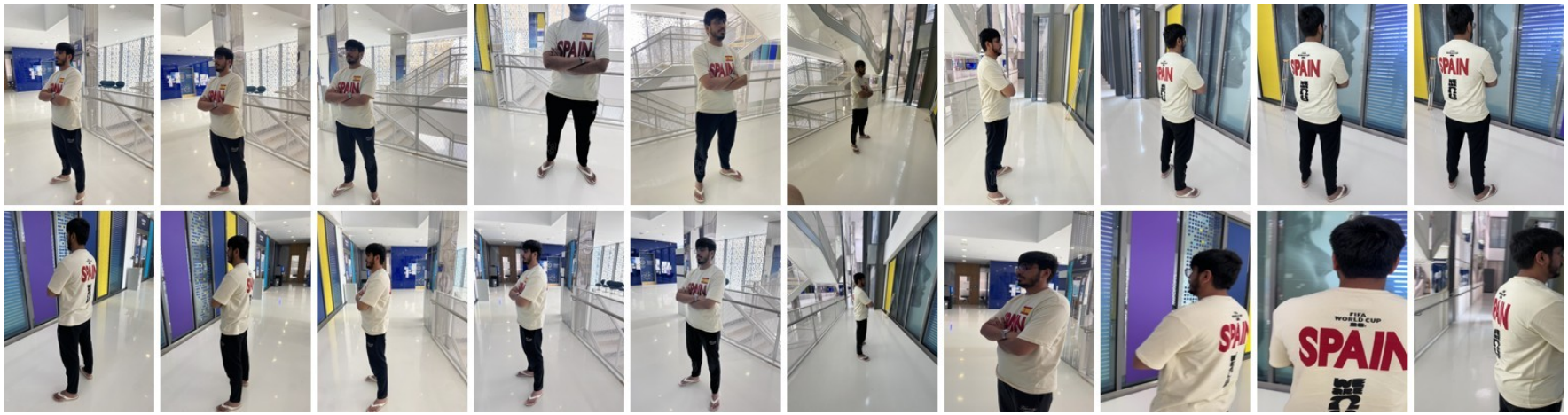




125 photos + their cameras (Assignment 1)

pictures from new viewpoints

Inputs



- The same 125 photos as Assignments 1 and 2 (4 iPhone lenses + video frames)
- Cameras from Assignment 1, undistorted to pinhole
- Person masks → person on a white background; resized to 1000 px
- 109 photos for training, 16 held out for scoring (every 8th, the 3DGS rule)
- No point cloud: **Gaussians start from random points**

Methods

Method	Venue	Year	Scene stored as
NeRF	**ECCV**	**2020**	**one neural network (8 layers × 256)**
TensoRF	**ECCV**	**2022**	**a grid split into planes and lines**
3D Gaussian Splatting (3DGS)	**ACM TOG / SIGGRAPH**	**2023**	**~150k coloured 3D blobs**
Mip-Splatting	**CVPR**	**2024**	**3DGS + two anti-aliasing filters**

Official code (NeRF: nerf-pytorch, the standard PyTorch version of the authors' TensorFlow code), same photos and cameras.
TensoRF, 3DGS, Mip-Splatting: 30,000 steps. NeRF: 60 minutes of training (the paper trains for 200,000+ steps).

3D Gaussian Splatting: how it works

official paper implementation

$$G(x) = \exp\left(-\tfrac{1}{2} \, \frac{(x-\mu)^2}{\sigma^2} \right)$$

$$G(x) = \exp\left(-\tfrac{1}{2} \, (x-\mu)^T \, \Sigma^{-1} \, (x-\mu) \right)$$

$$\mu_{cam} = R_c \, \mu + t_c, \qquad \Sigma_{cam} = R_c \, \Sigma \, R_c^T$$

3D Gaussian Splatting: how it works

official paper implementation

$$\mu_{cam} = R_c\,\mu + t_c, \qquad \Sigma_{cam} = R_c\,\Sigma\,R_c^T$$

$$u = f_x\,\frac{x}{z} + c_x, \qquad v = f_y\,\frac{y}{z} + c_y$$

$$\Sigma = R\,S\,S^T\,R^T$$

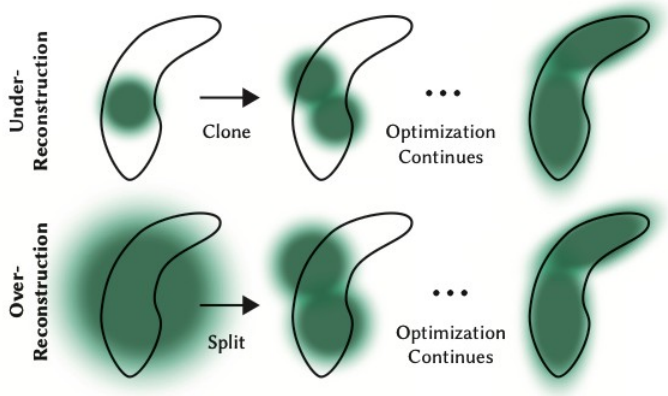
3D Gaussian Splatting: how it works

official paper implementation

$$\Sigma = R\,S\,S^T\,R^T$$

$$\Sigma' = J\,W\,\Sigma\,W^T\,J^T$$

$$C = \sum_{i=1}^{N} T_i \alpha_i \mathbf{c}_i,$$



3D Gaussian Splatting: how it works

official paper implementation

$$C = \sum_{i=1}^{N} T_i \alpha_i \mathbf{c}_i,$$

$$\alpha_i = (1 - \exp(-\sigma_i \delta_i))$$

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_i).$$

$$C = \sum_{i \in \mathcal{N}} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j),$$

3D Gaussian Splatting: how it works

official paper implementation

$$G(x) = \exp\Big(-\tfrac{1}{2} (x-\mu)^T \Sigma^{-1} (x-\mu) \Big)$$

$$\Sigma = R\, S\, S^T R^T$$

$$\mathcal{L} = (1-\lambda)\mathcal{L}_1 + \lambda \mathcal{L}_{\text{D-SSIM}}$$

$$\Sigma' = J\, W\, \Sigma\, W^T J^T$$

$$C = \sum_{i \in \mathcal{N}} c_i \alpha_i \prod_{j=1}^{i-1} (1-\alpha_j),$$

3D Gaussian Splatting: how it works

official paper implementation

$$G(x) = \exp\Big(-\tfrac{1}{2} (x-\mu)^T \Sigma^{-1} (x-\mu) \Big)$$

$$\mathcal{L} = (1-\lambda)\mathcal{L}_1 + \lambda \mathcal{L}_{\text{D-SSIM}}$$

$$\Sigma = R\,S\,S^T R^T$$

$$q = \left(\cos\tfrac{\theta}{2},\ \sin\tfrac{\theta}{2}\,a_x,\ \sin\tfrac{\theta}{2}\,a_y,\ \sin\tfrac{\theta}{2}\,a_z \right)$$

$$\Sigma' = J\,W\,\Sigma\,W^T J^T$$

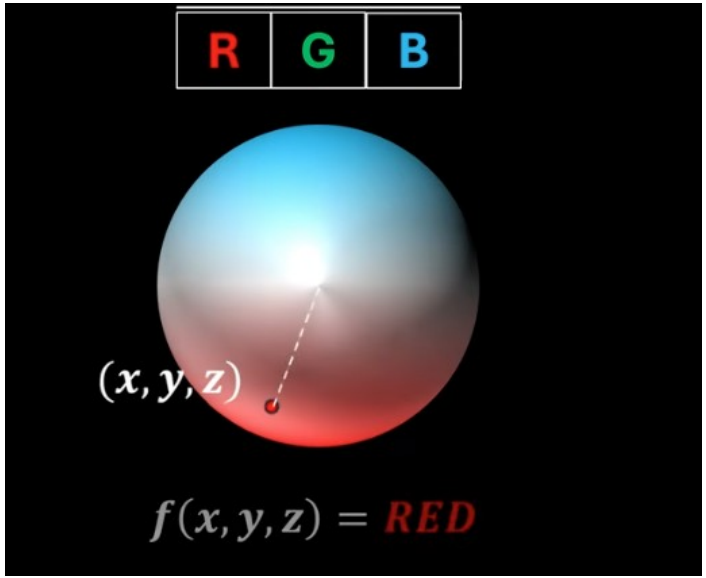
$$c(\mathbf{d}) = \sum_{l=0}^{L} \sum_{m=-l}^{l} k_l^m\, Y_l^m(\mathbf{d})$$

$$C = \sum_{i \in \mathcal{N}} c_i \alpha_i \prod_{j=1}^{i-1} (1-\alpha_j),$$

3D Gaussian Splatting: how it works

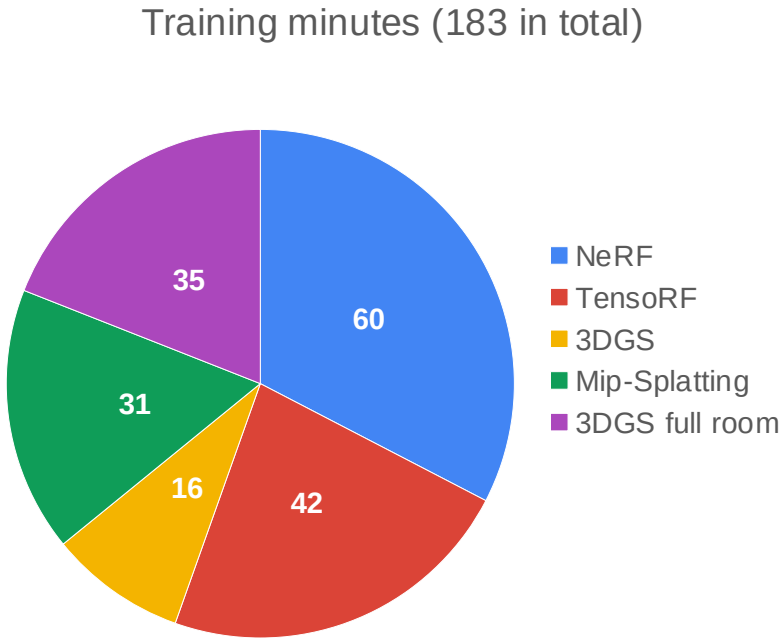
spherical harmonics

$$c(\mathbf{d}) = \sum_{l=0}^{L} \sum_{m=-l}^{l} k_l^m \, Y_l^m(\mathbf{d})$$



Experiments

- 5 trainings: NeRF, TensoRF, 3DGS, Mip-Splatting (person), 3DGS full room (extra)
- Scores on the 16 held-out photos: PSNR, SSIM, LPIPS
- Zoom test: render 3DGS and Mip-Splatting at ¼ size and 1.6× size
- Ablation: Gaussians from random points vs from the Assignment 1 sparse point cloud
- Result video: same 8 s orbit for every method
- I didn’t forget the official GUI render ;)



One RTX 5000 Ada (32 GB), trainings run side by side.

Results on held-out photos

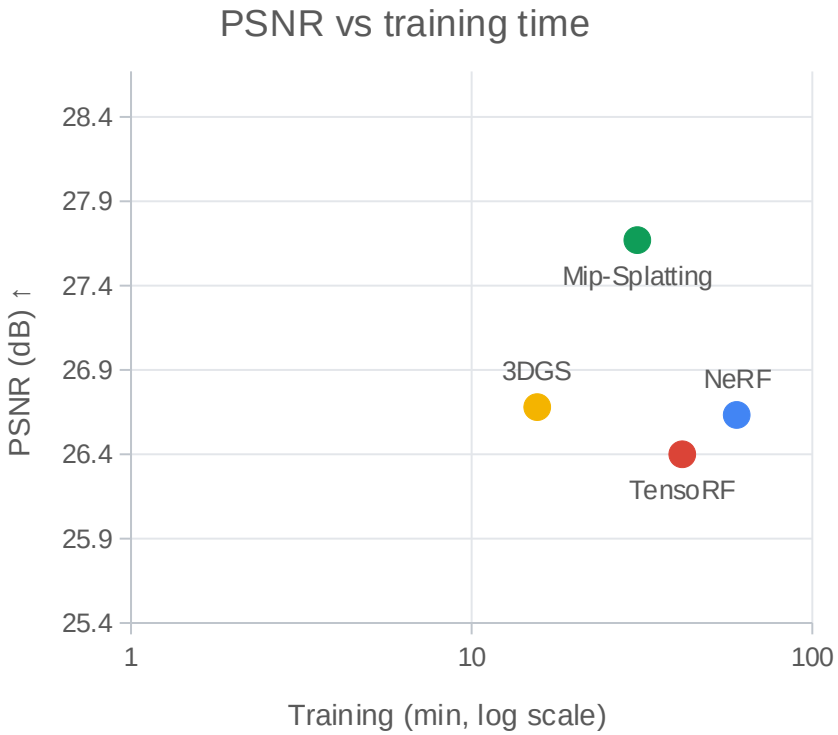


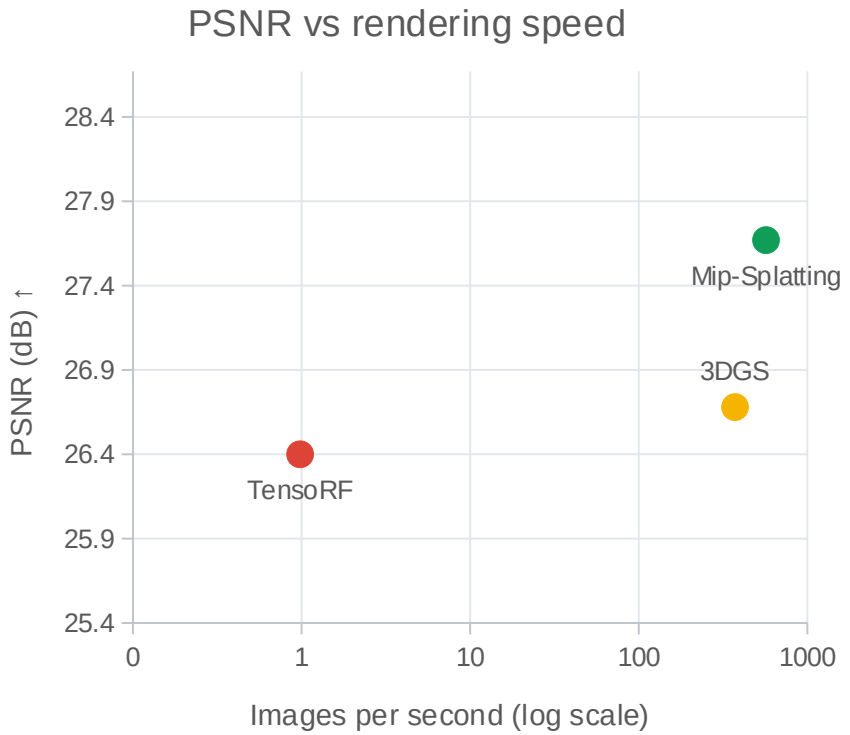
Comparison

best second best

Method	PSNR ↑	SSIM ↑	LPIPS ↓	Training ↓	FPS ↑	Primitives	Size (MB)
NeRF	26.59	0.931	0.132	60 min	0.04	1.2M	14
TensoRF	26.35	0.938	0.130	42 min	0.98	17.4M	67
3DGS	26.63	0.951	0.072	16 min	373	149k	35
Mip-Splatting	27.62	0.958	0.067	31 min	569	197k	47

Quality vs time and speed





Higher is better; left chart: further left is faster to train, right chart: further right draws faster.

Zoom test: 3DGS vs Mip-Splatting



Rendered at	3DGS	Mip
¼ size	23.71	28.63
1× (training)	26.63	27.62
1.6×	26.29	27.37

Left: rendered at ¼ size, enlarged ×6 (pixels kept square). Table: PSNR (dB), 16 held-out photos; ¼ = photo averaged down, 1.6× = original 1600 px photos.

Zoomed out, 3DGS blobs get thinner than a pixel and the image breaks up; Mip-Splatting stays clean.

Ablation: random start vs point cloud

Start	3DGS PSNR ↑	3DGS LPIPS ↓	Mip PSNR ↑	Mip LPIPS ↓	Room PSNR ↑
100k random points (main)	**26.63**	**0.072**	**27.62**	**0.067**	**17.96**
Assignment 1 point cloud	**27.53**	**0.064**	**27.53**	**0.064**	**19.71**

- Main results use only the photos and their cameras: every Gaussian starts at a random position
- The point cloud of Assignment 1 is only a better starting guess; training does the rest
- Same photos, cameras, 30,000 steps and scoring in both rows

Result videos (preset 8 s orbit)

NeRF

TensoRF

3DGS

Mip-Splatting



Frames 0, 40, 80, 120, 160, 200 of 240. The full videos are in results/videos/.

Extra: 3DGS with the room



- Same photos without the person masks: 17.96 dB PSNR on the held-out photos
- 0.91M Gaussians (person alone: ~150k), trained in 35 min
- The room is only seen from where the photos were taken, so views far from them break up

GUI: inside the course starter code



What the GUI's 3D view draws, from one held-out photo's camera (offscreen render of the same code)

Findings

- Best quality on held-out photos: Mip-Splatting (27.62 dB)
- Fastest to train: 3DGS (16 min); Gaussians draw in real time
- Mip-Splatting is best at every size; zoomed out by far (+4.9 dB)
- No point cloud needed: **from random points Mip-Splatting is even slightly better;** 3DGS loses ~0.9 dB
- Clean masks matter: the white background makes the person easy to learn
- NeRF: same rendering idea as TensoRF, but one big network → slowest to train and to draw

References

[1] B. Mildenhall et al. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. ECCV 2020.
[2] A. Chen, Z. Xu, A. Geiger, J. Yu, H. Su. TensoRF: Tensorial Radiance Fields. ECCV 2022.
[3] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM TOG 42(4), 2023.
[4] Z. Yu, A. Chen, B. Huang, T. Sattler, A. Geiger. Mip-Splatting: Alias-free 3D Gaussian Splatting. CVPR 2024.